



Démineur

PROJET EN JAVA – IMDS 4A

Table des matières

INTRODUCTION.....	2
DESCRIPTION.....	2
ARCHITECTURE DU CODE.....	3
PRESENTATION	5
<i>Diagrammes</i>	5
POINTS IMPORTANTS.....	8
DIFFICULTES RENCONTREES.....	8
AXES D'AMELIORATION.....	9
CONCLUSION	10

Introduction

Le but de ce projet est de créer une version fonctionnelle du jeu Démineur, un jeu classique où le joueur doit découvrir des cases sur une grille sans toucher aux mines. Ce projet a été réalisé dans le cadre du cours de programmation orientée objet, en utilisant Java pour mettre en œuvre une version console de ce jeu.

Le développement s'appuie sur un diagramme de classes et plusieurs diagrammes de séquence permettant de structurer l'architecture du programme avant son implémentation. Le jeu est réalisé en version console et respecte les règles classiques du démineur.

Description

Le démineur est un jeu de plateau constitué de cases. Le joueur doit découvrir les cases sans révéler de mine. Chaque case peut être :

- Vide : Aucune mine à proximité.
- Numérotée : Indique le nombre de mines autour de la case.
- Minée : Contient une mine.

Le joueur dispose de trois actions : Marquer une case pour signaler une mine supposée, découvrir une case, ou quitter la partie.

Si le joueur découvre une case minée, il perd la partie. Si toutes les cases sans mines sont découvertes, le joueur gagne. Si le joueur a marqué une mauvaise case, il peut revenir en arrière en marquant cette même case pour qu'elle devienne simplement couverte.

L'affichage console utilise les conventions suivantes :

- * : case couverte
- % : case marquée
- chiffre : case numérotée

- espace : case vide
- M : mine (en fin de partie)

Lorsque le joueur commence la partie, il choisit son niveau (Débutant, Intermédiaire, Avancé), qui va influencer sur la taille de la grille et le nombre de mines initiales. Le jeu commence ensuite et le nombre de mines restantes est affiché. Si le joueur découvre une case vide, toutes les cases vides autour sont découvertes jusqu'à tomber sur une case avec un numéro.

```

Nb mines restantes : 9
  0  1  2  3  4  5  6  7  8
0|
1|
2|      1  2  2  1
3|    1  1  3  *  *  1
4|    1  %  *  *  *  1
5|    1  1  2  *  *  2  3  2
6|      1  *  *  *  *  *
7|  1  1      1  *  *  *  *
8|  *  1      1  *  *  *  *

1 - Marquer une case
2 - Decouvrir une case
0 - Quitter
Choix : 2
Ligne : 4
Colonne : 3

```

Figure 1 : interface du jeu

Architecture du code

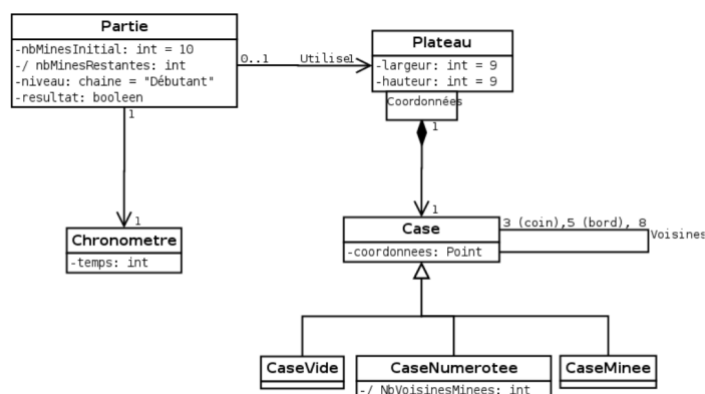


Figure 2 : Diagramme de classes d'analyse

L'architecture de l'application repose sur plusieurs classes principales :

- **Plateau** : Représente la grille du jeu. Elle initialise la grille, place les mines aléatoirement, calcule le nombre de mines adjacentes, relie chaque case à ses voisines, et gère les états des cases (découvertes, marquées, couvertes).
- **Partie** : Est le contrôleur principal du jeu. Elle gère l'état global (en cours, gagné, perdu), stocke le nombre de mines restantes, organise les interactions entre le joueur et le plateau.
- **Case** : Une classe abstraite représentant une case. Elle contient l'état courant des cases et la liste de leurs voisines. Ses 3 spécialisations (CaseVide, CaseNumerotee, et CaseMinee) gèrent respectivement les cases vides, les cases numérotées et les mines.
- **EtatCase et ses sous-classes** : représente comme son nom l'indique l'état des classes, basé sur un Design Pattern State. Il permet de gérer le comportement des cases sans conditions, ce qui rentre parfaitement dans l'orienté objet.

Voici plusieurs informations importantes du code à noter :

- **Singleton (dans Partie)** : Le jeu utilise un singleton pour garantir qu'il n'y ait qu'une seule instance de la partie en cours. Ce choix simplifie la gestion de l'état global du jeu.
- **Gestion des Mines** : Les mines sont placées aléatoirement sur le plateau. Une fois les mines placées, on calcule le nombre de mines adjacentes pour chaque case non minée.
- **Design Pattern State** : Chaque état gère son propre comportement (la réaction à découvrir() et à marquer()). Cela évite l'utilisation massive des conditions et facilite la lecture et modification du code.
- **Gestion des cases** : Les cases sont stockées dans une Map<Point, Case>, ce qui permet un accès direct via leurs coordonnées, sans dépendre d'indices fixes. Cette structure rend le code plus flexible si la taille du plateau doit évoluer dynamiquement.
- **Gestion des voisines** : Réalisé en parcourant les 8 directions autour de chaque case, tout en vérifiant les limites du plateau.

Présentation

Diagrammes

Les diagrammes UML suivants illustrent les principaux scénarios d'interaction du système.

Marquer une case

Pour marquer une case, l'utilisateur envoie un signal à la classe partie, qui elle-même envoie un signal à la classe Plateau. Celle-ci va récupérer la case correspondant aux coordonnées fournies et appeler la méthode appropriée. Elle va ensuite demander à la classe Case de la marquer, en fonction de son état. Le diagramme ci-dessous montre la gestion des différents états de la requête « Marquer ».

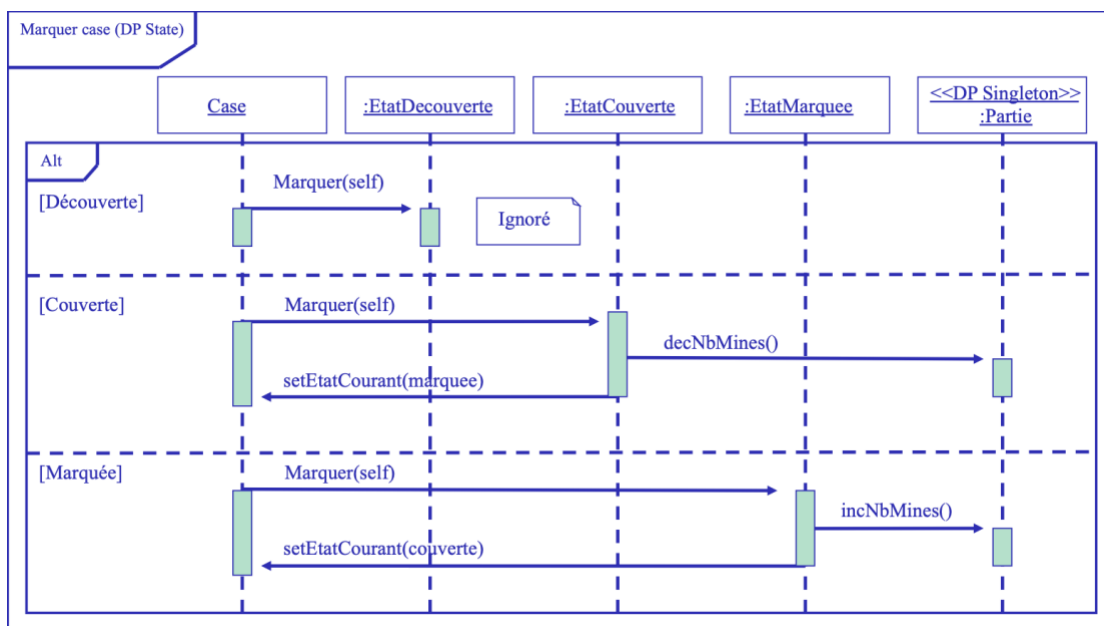


Figure 3 : Diagramme de séquence Marquer case

Lorsque l'état courant est modifié, comme lorsque l'on marque une case par exemple, EtatCourant dans Case est alors mis à jour. Cela permet de suivre l'avancée du jeu et de plus facilement le comparer aux cases couvertes. Si l'état d'une case en vient à être modifié, le nombre de mines augmente (ou diminue si l'on démarque une case), et met donc à jour la partie en cours.

Découvrir une case

Ce diagramme de séquence met en lumière le rôle central du State Pattern dans la gestion du comportement dynamique des cases. Lorsque l'utilisateur demande la découverte d'une case, la requête suit un chemin similaire à celui du marquage jusqu'à atteindre l'objet Case correspondant.

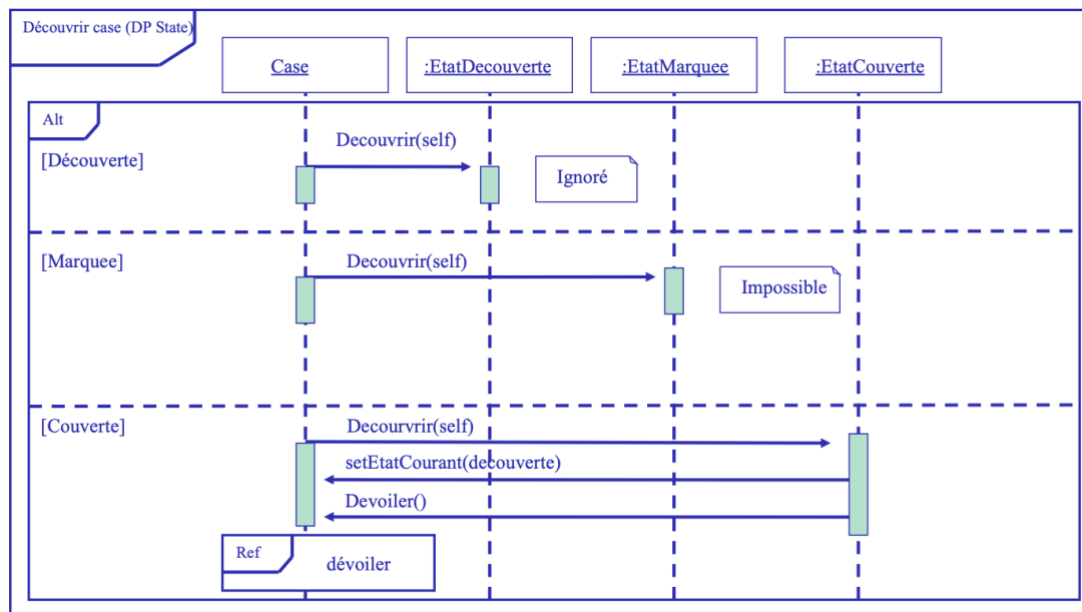


Figure 4 : Diagramme de séquence Découvrir case

Lorsque l'utilisateur souhaite découvrir une case :

- Si celle-ci est déjà découverte, alors l'action est ignorée.
- Si elle est marquée, l'action est aussi ignorée. Il faut la démarquer pour la découvrir.
- Si elle est couverte, on modifie alors l'état courant en EtatDecouverte et on dévoile la case au joueur.

Dévoiler une case

Ici, on met en évidence la logique interne du jeu, notamment la gestion des différents types de cases. Une fois l'état de la case modifié, la méthode `devoiler()` permet d'exécuter un comportement spécifique en fonction du type de case.

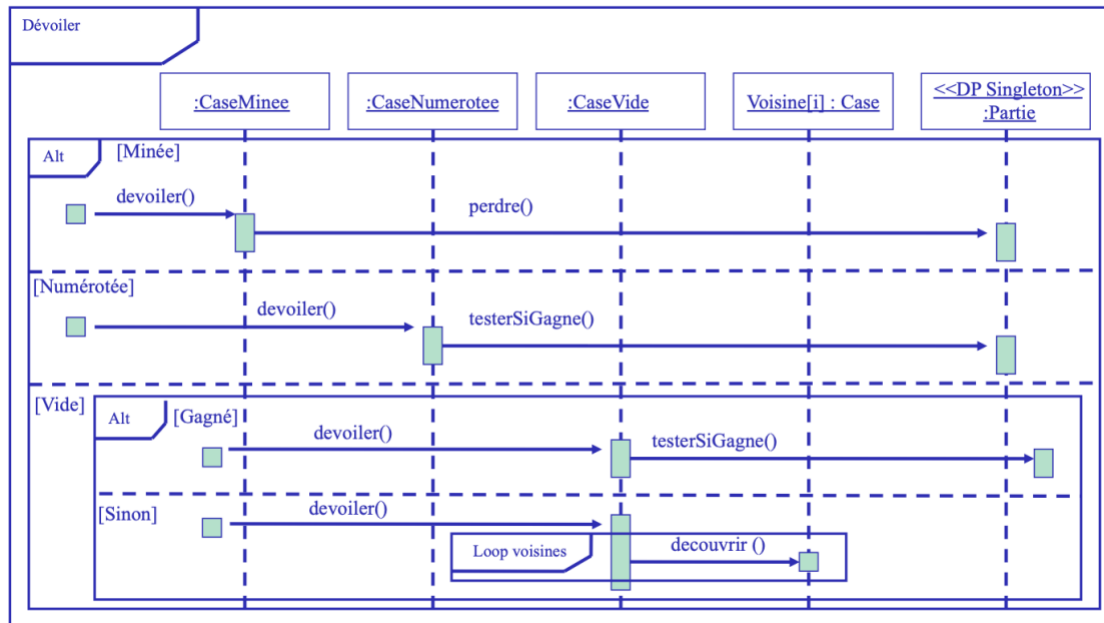


Figure 5 : Diagramme de séquence Dévoiler

Lorsque le joueur entre les coordonnées d'une case, celle-ci est découverte. Si la case est minée, le joueur perd la partie et toutes les mines sont affichées. La partie est finie.

Si la case est numérotée, alors on dévoile uniquement cette case et on teste si on a gagné ou non. Le joueur gagne une fois que toutes les cases non minées sont dévoilées.

Si la case est vide, elle dévoile toutes ses voisines de manière récursive grâce à une boucle sur la liste des voisines de chaque case.

Dans l'implémentation actuelle, les méthodes `decouvrir()` et `devoiler()` de la classe `Case` déclenchent le même comportement via l'état courant de la case. La distinction reste principalement conceptuelle et pourrait être renforcée par une séparation plus explicite des responsabilités dans les classes `EtatCase`.

Points importants

La séparation des responsabilités entre les classes permet de limiter le couplage et d'améliorer la maintenabilité du code. Chaque classe possède un rôle précis (gestion de la grille, gestion de la partie, gestion du comportement des cases), ce qui facilite la compréhension globale du système.

L'utilisation d'une structure de type Map pour stocker les cases permet un accès rapide aux coordonnées et garantit une gestion flexible de la grille.

Les classes abstraites permettent de définir un comportement commun tout en laissant la possibilité aux sous-classes de spécialiser leurs actions. Cela renforce la cohérence de la hiérarchie des classes.

Le polymorphisme est utilisé efficacement à travers la hiérarchie des classes Case. Les méthodes telles que afficher() ou dévoiler() sont redéfinies selon le type de case, permettant d'adapter le comportement sans modifier le code existant. Cela rend le programme plus extensible.

L'implémentation du Design Pattern State constitue un point fort majeur du projet. Il permet d'encapsuler les comportements liés aux états des cases et d'éviter l'utilisation de nombreuses conditions. Cette approche améliore la lisibilité du code et facilite l'ajout de nouveaux états si nécessaire.

L'architecture du projet est suffisamment générique pour permettre l'ajout de nouvelles fonctionnalités, comme de nouveaux types de cases ou de nouveaux modes de jeu, sans modifier en profondeur les classes existantes.

Difficultés rencontrées

Plusieurs difficultés techniques ont été rencontrées lors du développement.

Propagation récursive : La gestion de la révélation automatique des cases vides a posé problème. Il a fallu éviter les appels récursifs infinis en vérifiant que chaque case n'était révélée qu'une seule fois. Une mauvaise gestion de cette condition entraînait une boucle infinie.

Gestion des états : Maintenir la cohérence entre l'état logique d'une case et son affichage a nécessité une attention particulière. Le changement d'état devait systématiquement mettre à jour le nombre de mines restantes et éviter les transitions incohérentes.

Organisation des responsabilités : Il a fallu veiller à ne pas concentrer trop de logique dans une seule classe, notamment entre Partie et Plateau. Cette séparation a demandé plusieurs ajustements pour conserver une architecture claire.

Axes d'amélioration

Plusieurs améliorations pourraient être apportées au projet afin d'enrichir l'expérience utilisateur et de renforcer la robustesse du programme.

Tout d'abord, l'ajout d'exceptions permettrait de mieux gérer les erreurs liées aux entrées utilisateur, comme des coordonnées invalides ou des actions non autorisées, et d'éviter ainsi des comportements inattendus du programme.

Ensuite, l'intégration d'un chronomètre permettrait d'introduire une notion de performance et de comparer les parties entre elles. Cela rendrait le jeu plus compétitif et plus proche des versions classiques du démineur.

Enfin, une évolution majeure consisterait à ajouter une interface graphique, permettant une interaction plus intuitive, ainsi que la possibilité de choisir dynamiquement la taille du plateau et le nombre de mines.

Conclusion

Ce projet de réalisation du jeu Démineur en Java a permis de mettre en pratique de manière concrète les concepts fondamentaux de la programmation orientée objet. À travers la conception et l'implémentation du système, plusieurs notions clés ont été mobilisées, notamment l'abstraction, l'héritage, le polymorphisme et l'encapsulation.

L'utilisation des diagrammes UML a contribué à clarifier les interactions entre les classes et à anticiper les choix d'architecture, ce qui a rendu la phase d'implémentation plus fluide.

D'un point de vue personnel, ce travail a permis d'approfondir la compréhension des mécanismes de gestion d'état et de la conception orientée objet dans un contexte concret, tout en programmant en Java.

Enfin, ce projet constitue une base solide pouvant être enrichie par de futures améliorations, telles que l'ajout d'une interface graphique. Il illustre pleinement l'intérêt des principes de conception logicielle dans la réalisation d'une application fonctionnelle et évolutive.